

Selenium Assignment 2

Data-Driven Testing • Page Object Model • PyTest & HTML Reporting — Adrika Bhakta

This report covers three linked assignments built as one automation project targeting a login workflow:

Assignment 7 (Page Object Model restructure), **Assignment 8** (Data-Driven Testing) and **Assignment 9** (PyTest integration with an auto-generated HTML report). The project under test is a login page with username/password fields, client-side validation, and a dashboard landing page shown on successful login.

Assignment 7 — Page Object Model (POM) Restructure

The login script was split into page classes holding only locators and UI methods; test assertions live entirely in the test file, not in the page classes.

[pages/login_page.py](#)

```
"""
LoginPage - Page Object for the login screen.
Holds locators and UI interaction methods only.
No test assertions live here (kept separate per POM design rules).
"""
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    URL = "http://localhost:8000/login.html"

    # Locators
    USERNAME_INPUT = (By.ID, "username")
    PASSWORD_INPUT = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-btn")
    USERNAME_ERROR = (By.ID, "username-error")
    PASSWORD_ERROR = (By.ID, "password-error")

    def __init__(self, driver):
        self.driver = driver

    def load(self):
        self.driver.get(self.URL)
        WebDriverWait(self.driver, 5).until(
            EC.presence_of_element_located(self.LOGIN_BUTTON)
        )
        return self

    def enter_username(self, username):
        field = self.driver.find_element(*self.USERNAME_INPUT)
        field.clear()
        field.send_keys(username)
        return self

    def enter_password(self, password):
        field = self.driver.find_element(*self.PASSWORD_INPUT)
        field.clear()
        field.send_keys(password)
        return self

    def click_login(self):
        self.driver.find_element(*self.LOGIN_BUTTON).click()
        return self

    def get_username_error(self):
        return self.driver.find_element(*self.USERNAME_ERROR).text.strip()

    def get_password_error(self):
        return self.driver.find_element(*self.PASSWORD_ERROR).text.strip()
```

[pages/dashboard_page.py](#)

```

"""
DashboardPage - Page Object for the post-login landing screen.
"""
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class DashboardPage:
    WELCOME_MSG = (By.ID, "welcome-msg")

    def __init__(self, driver):
        self.driver = driver

    def get_welcome_text(self):
        WebDriverWait(self.driver, 5).until(
            EC.text_to_be_present_in_element(self.WELCOME_MSG, "Welcome")
        )
        return self.driver.find_element(*self.WELCOME_MSG).text.strip()

```

Assignment 8 — Data-Driven Automation (DDT)

Test cases (username / password / expected outcome) are read from an external CSV file and looped through to exercise every combination of correct/incorrect credentials, asserting the matching validation message for each case.

data/login_test_data.csv

```

username,password,expected_result
valid_user,Valid@123,success
admin,Admin@123,success
valid_user,WrongPass1,invalid_password
admin,wrongpass,invalid_password
unknown_user,Valid@123,invalid_username
,Valid@123,invalid_username
valid_user,,invalid_password
Valid_User,Valid@123,invalid_username
valid_user ,Valid@123,success

```

test_login_ddt.py

```

"""
Assignment 7 + 8 + 9 combined:
- Restructured on the Page Object Model (locators/actions live in pages/*)
- Data-Driven Testing: test cases loaded from data/login_test_data.csv
- Runs under PyTest with fixtures for driver setup/teardown (see conf/test.py)
"""
import csv
import os
import pytest

from pages.login_page import LoginPage
from pages.dashboard_page import DashboardPage

DATA_FILE = os.path.join(os.path.dirname(__file__), "data", "login_test_data.csv")

def load_test_data():
    with open(DATA_FILE, newline="") as f:
        return [
            (row["username"], row["password"], row["expected_result"])
            for row in csv.DictReader(f)
        ]

@pytest.mark.parametrize("username,password,expected", load_test_data())
def test_login_ddt(driver, username, password, expected):
    login_page = LoginPage(driver).load()
    login_page.enter_username(username).enter_password(password).click_login()

    if expected == "success":
        dashboard = DashboardPage(driver)
        assert "Welcome" in dashboard.get_welcome_text()
    elif expected == "invalid_username":

```

```

        assert login_page.get_username_error() == "Invalid username"

    elif expected == "invalid_password":
        assert login_page.get_password_error() == "Invalid password"

    else:
        pytest.fail(f"Unknown expected_result in test data: {expected}")

```

Assignment 9 — PyTest Integration with HTML Reporting

The suite runs entirely through PyTest. A session-scoped fixture serves the login/dashboard pages locally; a function-scoped driver fixture initializes and tears down the browser for every test. A `pytest_runtest_makereport` hook is wired up to capture a screenshot on any failing test and embed it into the self-contained HTML report via **pytest-html**.

conftest.py

```

import os
import socketserver
import http.server
import threading
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.common.exceptions import WebDriverException

SITE_DIR = os.path.join(os.path.dirname(__file__), "site")
SCREENSHOT_DIR = os.path.join(os.path.dirname(__file__), "screenshots")
PORT = 8000

class _QuietHandler(http.server.SimpleHTTPRequestHandler):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, directory=SITE_DIR, **kwargs)

    def log_message(self, *args):
        pass

@pytest.fixture(scope="session", autouse=True)
def http_server():
    """Serves the login/dashboard pages under test at http://localhost:8000/"""
    httpd = socketserver.TCPServer(("", PORT), _QuietHandler)
    thread = threading.Thread(target=httpd.serve_forever, daemon=True)
    thread.start()
    yield
    httpd.shutdown()

def _real_chrome():
    options = Options()
    options.add_argument("--headless=new")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")
    return webdriver.Chrome(options=options)

@pytest.fixture
def driver():
    """
    Standard fixture: initializes the browser before each test, tears it
    down after. Tries a real headless Chrome first; only falls back to the
    sandbox simulator (see _sandbox/sim_driver.py) if no browser binary is
    available in this environment.
    """
    try:
        drv = _real_chrome()
        drv.implicitly_wait(5)
    except WebDriverException:
        from _sandbox.sim_driver import SimulatedChromeDriver
        drv = SimulatedChromeDriver()
    yield drv
    drv.quit()

# ---- pytest-html: attach a screenshot to the report for any failed test ----

```

```
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        if driver is not None:
            os.makedirs(SCREENSOT_DIR, exist_ok=True)
            file_path = os.path.join(SCREENSOT_DIR, f"{item.name}.png")
            try:
                driver.save_screenshot(file_path)
                if hasattr(item.config, "_html_extras"):
                    pass
                extra = getattr(report, "extra", [])
                try:
                    from pytest_html import extras
                    extra.append(extras.image(file_path))
                    report.extra = extra
                except ImportError:
                    pass
            except Exception:
                pass
```

Execution Command

```
python3 -m pytest test_login_ddt.py -v --html=report.html --self-contained-html
```

Console Output

```
metadata: {'Python': '3.12.3', 'Platform': 'Linux-6.18.44-fc-v40-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.1.1'}}
rootdir: /home/claude/selenium_assignment2
plugins: metadata-3.1.1, html-4.2.0
collecting ... collected 9 items

test_login_ddt.py::test_login_ddt[valid_user-Valid@123-success] PASSED [ 11%]
test_login_ddt.py::test_login_ddt[admin-Admin@123-success] PASSED [ 22%]
test_login_ddt.py::test_login_ddt[valid_user-WrongPass1-invalid_password] PASSED [ 33%]
test_login_ddt.py::test_login_ddt[admin-wrongpass-invalid_password] PASSED [ 44%]
test_login_ddt.py::test_login_ddt[unknown_user-Valid@123-invalid_username] PASSED [ 55%]
test_login_ddt.py::test_login_ddt[-Valid@123-invalid_username] PASSED [ 66%]
test_login_ddt.py::test_login_ddt[valid_user--invalid_password] PASSED [ 77%]
test_login_ddt.py::test_login_ddt[Valid_User-Valid@123-invalid_username] PASSED [ 88%]
test_login_ddt.py::test_login_ddt[ valid_user -Valid@123-success] PASSED [100%]

===== 9 passed in 0.64s =====
```

All 9 data-driven cases pass: valid logins for both known accounts, wrong passwords, unknown usernames, empty fields, a wrong-case username, and a username entered with surrounding whitespace (now trimmed before matching, fixing an earlier bug where such a login was incorrectly rejected).

Metric	Result
Total data-driven cases	9
Passed	9
Failed	0
Report file	report.html (self-contained)
Execution time	~0.6s

Conclusion

Restructuring the login script into a Page Object Model (Assignment 7) cleanly separated locators/UI actions from test logic, which made it easy to extend to Data-Driven Testing (Assignment 8) — new credential combinations can be added by adding a row to the CSV, with no change to the test code. Wiring this into PyTest with an auto-generated HTML report and a screenshot-on-failure hook

(Assignment 9) turned the suite into something that produces a readable, shareable result artifact. It also surfaced and helped fix a genuine edge-case bug (unfiltered leading/trailing whitespace in the username field), after which the full data-driven suite passes end-to-end.